

Contents

Proyecto de Diseño Biomédico 2	1
□ Instrucciones de Gestión (LEER OBLIGATORIAMENTE)	1
□ Estructura del Repositorio (Design History File)	3
□ Resumen del Problema	4

Plantilla de tu Proyecto: Edita esta plantilla usando el ícono del lápiz y coloca la información de tu grupo.

Proyecto de Diseño Biomédico 2

Integrantes del Equipo: * Estudiante 1 * Estudiante 2 * Estudiante 3 * ...

Nombre del Proyecto: [Escribir aquí el nombre] **Stakeholder/Cliente:** [Nombre de la empresa, institución o usuario]

□ Instrucciones de Gestión (LEER OBLIGATORIAMENTE)

Click aquí para expandir

Para la evaluación de este curso, utilizamos la metodología “**Si no está en el Issue, no existe**”.

Este curso evalúa desempeño **individual** dentro del equipo. Este repositorio se creó a partir del usuario de GitHub que cada integrante reportó a su profesor el día de la conformación de grupos, ese usuario es tu identidad de trabajo aquí, y todo lo que subas queda registrado con él. Haz tus propios commits, con tu propia cuenta: no subas evidencia de un compañero “de favor” con tu usuario, porque le quitas la trazabilidad de su trabajo. Si cambiaste de usuario de GitHub después de esa clase, avísale a tu profesor para que ajuste tu invitación.

1. Configuración Inicial (Semana 1)

1. Vayan a la pestaña **Projects** de este repositorio y creen un **New Project** con la vista **Table**.
2. Al crearlo, GitHub les ofrece **importar los Issues del repositorio directamente a la tabla**, acéptenlo, así no hay que agregarlos uno por uno.
3. Agreguen la columna **Milestone** (no aparece por defecto): botón **+** al final de las columnas de la tabla.
4. Dejen visibles solo estas columnas y oculten el resto: **Title, Assignees, Status, Milestone**.
5. Guarden esta vista con el botón **View → Save view**, para que no se pierda si alguien la cierra o recarga la página.
6. Para cada tarea, asignen un **Responsable** (columna Assignees), el 0.1 de cada integrante ya viene autoasignado.

2. Cómo entregar tareas

El profesor **NO revisará** carpetas al azar buscando archivos. El profesor revisará el **Project Board**. * Cada tarea completada debe tener un **Issue cerrado**. * Para cerrar un Issue válidamente, deben vincular la evidencia: * **Si es código/diseño:** En el commit message usen `fixes #numero_issue`. * **Si es documento/foto:** Súbanlo primero a la carpeta correspondiente del repositorio (commit + push), y luego enlázelo en un comentario del Issue con notación

Markdown [nombre del archivo](ruta/al/archivo), y ciérrerlo. No lo arrastren directo al comentario del Issue: eso lo sube dos veces (una al Issue, otra al repo) y el profesor termina revisando el repositorio, no el adjunto del Issue. (*¿No conoces esta notación de Markdown ni cómo copiar la ruta/el link de un archivo en GitHub? Ver la sección de recursos al final de este documento.*)

3. Formato de la documentación: Markdown, no binarios

Como la evaluación es individual, la documentación de equipo (informes, actas, matrices de selección, bitácoras, justificación ingenieril) se escribe en archivos de texto **Markdown (.md)** dentro del repositorio, **no** en Word, Google Docs ni OneDrive compartido. * **¿Por qué?** Un .md editado y comiteado por cada persona deja un historial claro de quién escribió qué y cuándo. Un documento compartido en Drive/OneDrive NO tiene esa trazabilidad: para la evaluación individual, ese trabajo “no existe” aunque el equipo sí lo haya hecho. * Pueden usar Google Docs para lluvia de ideas o borradores rápidos, pero el entregable final que se evalúa debe quedar como .md en el repositorio. * Esta restricción aplica solo a la **redacción colaborativa de documentos**. Los archivos binarios (CAD, esquemáticos, PCB, simulaciones, fotos, video) sí se suben tal cual, en las carpetas de 03_Ingenieria_Tecnica y 04_Multimedia_Prototipo descritas abajo. * Para insertar una imagen dentro de un .md (ej. una foto de avance o un boceto), arrástrala directamente al editor de texto de GitHub, se sube sola y queda enlazada en el archivo.

4. Trabajar juntos en videollamada (sin perder la autoría)

Si están acostumbrados a abrir un documento de Google Docs/OneDrive en videollamada y construirlo hablando entre todos, pueden lograr lo mismo con .md así: * **Dividan el documento por secciones o por archivos, cada uno con un dueño.** Ej.: estado_del_arte.md lo escribe una persona, requerimientos.md otra. En la llamada discuten y construyen el contenido juntos en tiempo real, pero cada quien escribe y comitea su propia parte desde su propio computador. * Así la conversación es simultánea (como en Drive), pero cada commit sigue siendo 100% de una sola persona, sin eso, la evaluación individual no se puede sostener. * Existen herramientas de edición simultánea “estilo Google Docs” para código (ej. VS Code Live Share), pero las descartamos para este curso: por defecto todo el texto tecleado en la sesión termina comiteado a nombre de quien controla la sesión, así que no resuelven el problema de trazabilidad individual sin disciplina extra, es más complejidad sin beneficio real sobre la opción de “sección con dueño”.

5. Recursos: Markdown y enlaces en GitHub

Si no estás familiarizado con Markdown (la notación [nombre](ruta), encabezados, listas) ni con las opciones de enlace que ofrece GitHub (ej. el ícono de eslabón  que aparece al pasar el mouse sobre un encabezado en un .md, o “Copy permalink” en un archivo de código): * **Basic writing and formatting syntax** : la referencia oficial de GitHub sobre Markdown, incluyendo la sección “Relative links” (justo lo que necesitas para enlazar archivos del repositorio). * **Communicate using Markdown** : curso interactivo gratuito de GitHub Skills (menos de 1 hora): practicas Markdown directamente en un repositorio de prueba, con retroalimentación paso a paso.

6. Recursos Recomendados para Dominar GitHub (Sin sufrir)

GitHub puede parecer intimidante al principio. No se preocupen, aquí tienen herramientas divertidas y sencillas para aprender en tiempo récord:

La Herramienta Salvavidas (Descárguenla ya! Recomendadísimo):

GitHub Desktop: Si la consola de comandos les da miedo, **usen esto**. Es la aplicación visual oficial. Permite subir cambios, ver el historial y sincronizar archivos arrastrando y soltando, sin escribir comandos oscuros. **Altamente recomendada para manejar los archivos CAD y Documentos.**

1. Para aprender jugando (Literalmente):

● **Oh My Git!:** Un juego open-source increíble. Convierte los comandos de Git en cartas de juego. Es visual, divertido y perfecto para entender la lógica sin escribir una sola línea de código al principio.

● **Learn Git Branching:** Es el estándar de oro para aprender visualmente. Ves gráficamente cómo se crean las ramas y los nodos cada vez que haces algo. Tiene niveles y es muy interactivo.

2. El Tutorial “A prueba de balas” (En 5 minutos):

● **Git - La guía sencilla:** Como su nombre lo indica, es una sola página, muy visual y sin palabras complicadas. “Git para seres humanos”.

3. Aprender dentro del mismo GitHub (Con un Robot):

● **GitHub Skills: Introduction:** Este es un curso oficial de GitHub. Al iniciarlo, un “bot” crea un repositorio real para ti y te va guiando paso a paso (te pide que hagas un cambio, que abras un Issue, etc.) y te corrige automáticamente. ¡Es aprender haciendo!

7. Consolidar avances en PDF

Al cierre de un hito (o para armar el borrador del reporte final ABET), pueden combinar varios .md del repositorio en un solo PDF, sin instalar nada en su computador:

1. Vayan a la pestaña **Actions** → “**Generar PDF desde Markdown**” → **Run workflow**.
2. En **archivos**, escriban las rutas de los .md a combinar, en el orden que quieren que aparezcan, separadas por espacio (ej. los de una Fase del 02_DHF para un hito, o varias fases juntas para el borrador del reporte final).
3. En **salida**, escriban la ruta y nombre del PDF resultante (ej. 01_Gestion_del_Proyecto/Consolidado_Hito1).
4. Denle **Run workflow**. En un par de minutos, el PDF queda commiteado en esa ruta del repositorio — no se pierde, queda como parte del historial del proyecto.

Esto no reemplaza el reporte final (que se arma con la plantilla institucional según el checklist ABET), pero les da una base ya ordenada y en un solo documento para partir de ahí.

□ Estructura del Repositorio (Design History File)

Este repositorio organiza el proyecto separando la evidencia documental de los archivos técnicos de ingeniería.

- **/01_Gestion_del_Proyecto:** Cronograma, Presupuesto, Licencias, Cartas de intención, Acuerdos legales,...
- **/02_DHF_Proceso_Diseño: DOCUMENTACIÓN (INFORMES Y EVIDENCIA)**
 - *Fase_1_Investigacion:* Definición de requerimientos, normas y estado del arte, etc
 - *Fase_2_Conceptual:* Matrices de selección, bocetos y definición de arquitectura, etc

- *Fase 3_Diseño_Detalle*: **Justificación ingenieril**. Aquí van los informes de cálculos, resultados de simulaciones, BOM final y especificaciones técnicas.
 - *Fase 4 Validación*: Protocolos de pruebas, resultados de validación con usuarios, análisis estadístico, etc
 - **/03_Ingeniería_Técnica: ARCHIVOS FUENTE EDITABLES (HERRAMIENTAS)**
 - *Calculos_Simulaciones*: Archivos de Excel, Scripts (MATLAB/Python), Simulaciones (Ansys/Proteus), simulaciones de SolidWorks, etc.
 - *Mecánica_CAD*: Archivos nativos de modelado 3D y planos de fabricación.
 - *Electrónica*: Esquemáticos, diseño de PCB (Gerbers) y listas de materiales.
 - *Software_Firmware*: Código fuente documentado.
 - **/04_Multimedia_Prototipo: MATERIAL DE DIVULGACIÓN.**
 - Fotos de alta calidad del prototipo final.
 - Video de funcionamiento (Demo) para la presentación final.
 - Renders comerciales o material para el Póster/Pitch.
 - **/05_Reporte_Final_ABET:**
 - Aquí se entregará el documento final compilado (“Capstone Project”) siguiendo la plantilla institucional.
 - Este documento debe alimentarse de la información generada en la carpeta 02_DHF.
-

□ Resumen del Problema

(*Editar esta sección en la Semana 2*) Describa brevemente en 1 párrafo cuál es la necesidad clínica o problema que están resolviendo.